Práctica 1 de Extensiones de Machine Learning
Análisis experimental de métodos de aprendizaje por refuerzo: bandido de k-brazos, control tabular y métodos aproximados

Aída García Echavarría
aida.g@um.es

Christian Andrés Rueda Ayala
candres.rueda@um.es

Pablo Daniel Cuña Cabrera
pablodaniel.cuna@um.es

15 de marzo de 2026

Resumen

En este trabajo se aborda el problema del aprendizaje por refuerzo mediante la implementación, análisis y comparación de distintos métodos en escenarios de complejidad creciente. En una primera parte, se estudia el problema del bandido de k-brazos, comparando las estrategias ε-greedy, Softmax y UCB sobre distribuciones de recompensa Normal, Bernoulli y Binomial. Posteriormente, se analiza el aprendizaje en entornos complejos, distinguiendo entre métodos tabulares y métodos aproximados.

En el bloque de métodos tabulares se evalúan Monte Carlo *on-policy*, Monte Carlo *off-policy*, SARSA y Q-Learning sobre el entorno `Taxi-v3`. Los resultados muestran que los métodos basados en diferencias temporales alcanzan el mejor desempeño global, con políticas eficaces, episodios cortos y tasas de éxito del 100 % en evaluación *greedy*, mientras que los enfoques Monte Carlo presentan una convergencia más lenta y un rendimiento final inferior. En particular, Q-Learning obtiene el mejor resultado dentro de este grupo.

En el bloque de métodos aproximados se comparan SARSA semi-gradiente y Deep Q-Network (DQN) en los entornos `CartPole-v1` y Tetris. En `CartPole-v1`, SARSA semi-gradiente supera a DQN tanto en estabilidad como en rapidez de convergencia, alcanzando el retorno máximo teórico del entorno. En Tetris, en cambio, ambos métodos obtienen un desempeño final prácticamente equivalente en evaluación *greedy*. En conjunto, los experimentos muestran que la elección del método depende no solo de la complejidad del entorno, sino también de la representación del estado, de la estabilidad del aprendizaje y del equilibrio entre simplicidad y capacidad de generalización.

Índice general

1. Problema del Bandido de k-brazos **3**

1.1. Introducción . . . 3

1.2. Desarrollo . . . 3

1.2.1. Definición formal del problema . . . 3

1.2.2. Trabajos Relacionados . . . 3

1.3. Algoritmos . . . 4

1.3.1. ε-greedy . . . 4

1.3.2. Softmax . . . 4

1.3.3. UCB . . . 4

1.4. Evaluación y Experimentos . . . 5

1.4.1. Configuración experimental . . . 5

1.4.2. Métricas . . . 5

1.4.3. Resultados comparativos . . . 5

1.5. Conclusiones . . . 7

2. Aprendizaje en Entornos Complejos **8**

2.1. Introducción . . . 8

2.2. Desarrollo . . . 8

2.2.1. Definición formal del problema . . . 8

2.2.2. Métodos tabulares . . . 9

2.2.3. Métodos aproximados . . . 9

2.2.4. Trabajos Relacionados . . . 9

2.3. Algoritmos . . . 9

2.3.1. Algoritmos tabulares . . . 9

2.3.2. Algoritmos aproximados . . . 11

2.4. Evaluación/Experimentos . . . 12

2.4.1. Métodos tabulares . . . 12

2.4.2. Métodos aproximados 14
2.5. Conclusiones 17

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

Al explicar el problema del bandido de k-brazos se hace una analogía con una máquina tragamonedas. Si nos encontramos frente a este tipo de máquinas, hay distintos botones, cada uno asociado a una acción y, por lo tanto, a una distribución de recompensa desconocida. Cada vez que una persona juega en esta máquina, elige un botón y recibe una recompensa. El objetivo es obtener la máxima recompensa acumulada posible a lo largo del tiempo. Este enfoque tiene numerosas aplicaciones prácticas, como la publicidad online, las promociones en comercio electrónico, los sistemas de recomendación o la toma de decisiones en el ámbito médico [1].

El estudio del problema del bandido de k-brazos resulta fundamental para comprender los conceptos de exploración y explotación, que constituyen una base del Aprendizaje por Refuerzo y de algoritmos más complejos.

El objetivo de este trabajo es profundizar en el problema del bandido de k-brazos mediante la implementación y comparación de distintas políticas de selección de acciones a través de diversas métricas de evaluación. Para ello, se consideran tres estrategias: ε-greedy, Softmax y UCB, aplicadas sobre un problema de bandido de k-brazos con $k = 10$. Cada una de estas estrategias se evalúa bajo tres tipos de distribuciones de recompensa: Bernoulli, Binomial y Normal, con el fin de comparar su comportamiento en distintos entornos probabilísticos.

Este capítulo se organiza de la siguiente manera. En la sección Desarrollo se presenta la formulación del problema del bandido de k-brazos y el marco conceptual necesario para su comprensión, incluyendo una breve revisión de trabajos relacionados. A continuación, en la sección Algoritmos se describen las estrategias de selección de acciones implementadas. Posteriormente, en la sección Evaluación y Experimentos se detalla la configuración experimental adoptada y se analizan los resultados obtenidos considerando distintas distribuciones de recompensa. Finalmente, en la sección Conclusiones se sintetizan los principales hallazgos derivados del estudio.

1.2. Desarrollo

En esta sección se presenta el problema del bandido de k-brazos de manera más formal y se introducen algunos trabajos relacionados que permiten contextualizar su estudio.

1.2.1. Definición formal del problema

El problema del bandido de k-brazos consiste en un conjunto finito de acciones o brazos disponibles, denotado por $A = \{1, 2, \ldots, k\}$. Cada brazo i se asocia a una distribución de probabilidad desconocida con media μ_i.

En cada instante $t = 1, \ldots, T$, el agente selecciona un brazo A_t y recibe una recompensa R_t generada según la distribución asociada a dicho brazo. El objetivo es maximizar la recompensa acumulada a lo largo del horizonte temporal:

$$\sum_{t=1}^{T} R_t \tag{1.1}$$

De manera equivalente, se busca minimizar el *regret* acumulado, definido como:

$$\text{Regret}(T) = \sum_{t=1}^{T} (\mu^* - \mu_{A_t}) \tag{1.2}$$

donde $\mu^* = \text{máx}_i \, \mu_i$ representa la media del brazo óptimo. Esta formulación pone de manifiesto el dilema entre exploración y explotación, ya que el agente debe decidir entre probar brazos poco conocidos o explotar aquellos que han mostrado mejor desempeño [2].

Además de estas magnitudes teóricas, en la evaluación experimental de las políticas consideradas se emplean métricas complementarias que permiten analizar su capacidad para identificar y seleccionar el brazo óptimo a lo largo del tiempo.

1.2.2. Trabajos Relacionados

El problema del bandido de k-brazos ha sido ampliamente estudiado en la literatura como una formulación fundamental del dilema exploración–explotación dentro del Aprendizaje por Refuerzo. Sutton y Barto [2] lo presentan como uno de los problemas más representativos para introducir estrategias de selección de acciones basadas en estimaciones de valor. Por su parte, Auer et al. [3] desarrollan un análisis finito del problema y proponen políticas basadas en cotas de confianza, entre ellas UCB, mostrando garantías teóricas en términos de *regret*. Más recientemente, Lattimore y Szepesvári [1] ofrecen un tratamiento integral del problema de los *bandits*, reuniendo en un mismo marco distintas familias de algoritmos, sus propiedades teóricas y sus aplicaciones más relevantes.

Además de su interés teórico, este tipo de problemas ha sido utilizado en aplicaciones reales. Por ejemplo, Li et al. [4] modelan la recomendación personalizada de noticias como un problema de *bandits* contextuales y muestran mejoras en la tasa de clics en un entorno real. Asimismo, Bouneffouf y Rish [5] presentan una revisión de aplicaciones prácticas de los *bandits* en dominios como recomendación, recuperación de información y salud, mientras que Villar et al. [6] revisan su uso en el diseño adaptativo de ensayos clínicos.

En este contexto, el presente trabajo no propone un nuevo algoritmo, sino una comparación experimental y sistemática de distintas estrategias clásicas de selección de acciones bajo distribuciones de recompensa de diferente naturaleza. A diferencia de los trabajos aplicados, aquí se considera un entorno controlado que permite aislar el efecto de cada política y analizar de forma homogénea su comportamiento mediante un mismo protocolo experimental.

1.3. Algoritmos

En esta sección se presentan los algoritmos empleados para resolver el problema del bandido de k-brazos. Se describen las estrategias ε-greedy, Softmax y UCB, destacando la lógica de selección de acciones en cada una de ellas.

1.3.1. ε-greedy

El algoritmo ε-greedy equilibra exploración y explotación mediante un parámetro $\varepsilon \in [0, 1]$.

En cada paso:

- Con probabilidad $1 - \varepsilon$, selecciona el brazo con mayor valor estimado.
- Con probabilidad ε, selecciona un brazo aleatoriamente.

En la implementación considerada, antes de aplicar esta regla se garantiza que todos los brazos sean seleccionados al menos una vez, con el fin de disponer de una estimación inicial para cada uno de ellos.

La estimación del valor esperado de cada brazo se actualiza mediante promedio incremental:

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N_t(a)}(R_t - Q_t(a)) \tag{1.3}$$

donde $N_t(a)$ representa el número de veces que el brazo a ha sido seleccionado.

La lógica general del procedimiento descrito se resume en el Algoritmo 1.

Algorithm 1 ε-greedy

```
for t = 1 to T do
    Generate random number u ~ U(0,1)
    if u < ε then
        Select random arm
    else
        Select arm with highest estimated value
    end if
    Observe reward R_t
    Update estimate of selected arm
end for
```

1.3.2. Softmax

El algoritmo Softmax equilibra exploración y explotación mediante una política probabilística basada en los valores estimados de cada brazo. A diferencia de ε-greedy, no selecciona acciones aleatorias de manera uniforme, sino que asigna a cada brazo una probabilidad de selección en función de su valor estimado.

La probabilidad de seleccionar el brazo a en el instante t se define mediante una distribución Softmax o de Boltzmann:

$$P_t(a) = \frac{\exp\left(Q_t(a)/\tau\right)}{\sum_{b=1}^{k} \exp\left(Q_t(b)/\tau\right)} \tag{1.4}$$

donde $Q_t(a)$ es el valor estimado del brazo a y $\tau > 0$ es el parámetro de temperatura. Valores pequeños de τ favorecen la explotación, concentrando la probabilidad en los brazos con mayor valor estimado, mientras que valores grandes promueven una exploración más uniforme.

En la implementación considerada, las probabilidades se calculan a partir de los valores estimados y se aplica una estabilización numérica previa a la exponencial para evitar desbordamientos. Luego, el brazo se selecciona muestreando de acuerdo con la distribución obtenida.

La estimación del valor esperado de cada brazo se actualiza mediante el mismo esquema de promedio incremental definido en la Ecuación 1.3, donde $N_t(a)$ representa el número de veces que el brazo a ha sido seleccionado.

La lógica general de esta estrategia se resume en el Algoritmo 2.

Algorithm 2 Softmax

```
for t = 1 to T do
    Compute action probabilities using Softmax over estimated values
    Select arm according to the probability distribution
    Observe reward R_t
    Update estimate of selected arm
end for
```

1.3.3. UCB

Las estrategias UCB (*Upper Confidence Bound*) abordan el dilema exploración–explotación incorporando, además del valor estimado de cada brazo, un término de incertidumbre que favorece la exploración de acciones menos probadas. En este trabajo se consideran las variantes UCB1 y UCB2.

En UCB1, la acción seleccionada en el instante t se obtiene maximizando la suma entre el valor estimado del brazo y un término de exploración:

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right] \tag{1.5}$$

donde $Q_t(a)$ es el valor estimado del brazo a, $N_t(a)$ representa el número de veces que dicho brazo ha sido seleccionado hasta el instante t, y $c > 0$ controla la intensidad de la exploración. En la implementación utilizada, primero se garantiza que todos los brazos sean seleccionados al menos una vez y luego se aplica esta regla. La estimación del valor esperado se actualiza mediante el mismo esquema de promedio incremental definido en la Ecuación 1.3. La lógica general de esta variante se resume en el Algoritmo 3.

Algorithm 3 UCB1

```
for t = 1 to T do
    if there exists an arm not yet selected then
        Select that arm
    else
        Select arm that maximizes upper confidence bound
    end if
    Observe reward R_t
    Update estimate of selected arm
end for
```

La variante UCB2 mantiene la misma idea general, pero organiza la exploración por épocas, de modo que el brazo seleccionado se repite durante varias iteraciones antes de reconsiderar la decisión. Para ello, define

$$\tau(r) = \lceil (1+\alpha)^r \rceil \tag{1.6}$$

y utiliza el término

$$a(n,r) = \sqrt{\frac{(1+\alpha)\ln\left(\frac{en}{\tau(r)}\right)}{2\tau(r+1)}} \tag{1.7}$$

con el que se construye el índice

$$UCB2(i) = Q(i) + a(n, r_i) \tag{1.8}$$

donde r_i es el número de épocas completadas para el brazo i. Una vez elegido, dicho brazo se mantiene durante $\tau(r_i + 1) - \tau(r_i)$ iteraciones. En la implementación utilizada, también se asegura inicialmente que todos los brazos sean seleccionados al menos una vez, y el efecto del parámetro α se analiza posteriormente en la sección experimental.

1.4. Evaluación y Experimentos

En esta sección se describen la configuración experimental y las métricas utilizadas, así como los resultados más relevantes obtenidos en la comparación de las estrategias consideradas. Debido a las restricciones de espacio del informe, se presentan únicamente los resultados comparativos finales y aquellos elementos que se consideran más representativos del análisis, quedando el detalle completo de los experimentos y su implementación disponible en el repositorio del proyecto.[1]

1.4.1. Configuración experimental

Los experimentos se realizaron sobre un problema de bandido de k-brazos con $k = 10$, considerando un horizonte temporal de $T = 1000$ pasos por ejecución y promediando 500 corridas independientes. En todos los casos se utilizó semilla fija ($seed = 42$) para asegurar reproducibilidad. Se consideraron tres tipos de distribuciones de recompensa: Normal, Bernoulli y Binomial con parámetro $n = 10$.

En ε-greedy, se evaluaron los valores $\varepsilon \in \{0,0{,}01, 0{,}1\}$, con el objetivo de estudiar cómo varía el equilibrio entre exploración aleatoria y explotación. Para Softmax, se analizaron temperaturas $T \in \{0{,}1, 0{,}5, 1{,}0\}$, buscando observar cómo el parámetro de temperatura modifica la distribución de probabilidad sobre las acciones y, en consecuencia, el grado de exploración de la política. En la familia UCB se consideraron dos variantes. Para UCB1 se probaron los valores $c \in \{1, \sqrt{2}, 2\}$, con el fin de evaluar el efecto del peso del término de confianza. Para UCB2 se analizaron $\alpha \in \{0{,}1, 0{,}5, 0{,}9\}$, estudiando cómo la duración de las épocas de exploración influye en el desempeño del algoritmo.

1.4.2. Métricas

Para evaluar el desempeño de las políticas consideradas se utilizaron tres métricas principales. La primera es la recompensa promedio obtenida a lo largo del tiempo, que permite observar la capacidad de cada estrategia para acumular retornos favorables durante el aprendizaje. La segunda es el porcentaje de selección del brazo óptimo, que mide con qué frecuencia la política identifica y explota la mejor acción disponible. Finalmente, se consideró el *regret* acumulado, que cuantifica la diferencia entre la recompensa esperada del brazo óptimo y la recompensa asociada a las acciones efectivamente seleccionadas.

De estas métricas, el *regret* acumulado se considera la más representativa, ya que permite medir de forma directa la eficiencia del proceso de aprendizaje, tal como se definió en la Ecuación 1.2.

1.4.3. Resultados comparativos

A partir de los experimentos individuales realizados para cada familia de algoritmos, se seleccionaron las configuraciones que mostraron mejor desempeño en cada distribución de recompensa. En el caso de UCB, se evaluaron las variantes UCB1 y UCB2; sin embargo, los resultados de UCB1 no se incluyen en esta sección, ya que en la comparación interna de la familia UCB la variante UCB2 mostró un mejor desempeño global. Por este motivo, la comparación final entre políticas se realizó utilizando UCB2. La Tabla 1.1 resume las configuraciones elegidas para dicha comparación.

Sobre estas configuraciones se realizó la comparación final utilizando las tres métricas definidas en la sección anterior. Los valores obtenidos se presentan en la Tabla 1.2. En términos

[1] https://github.com/imchrisrueda/extML/tree/master

Tabla 1.1: Mejores configuraciones seleccionadas para la comparación final

Distribución	ε-greedy	Softmax	UCB2
Normal	$\varepsilon = 0{,}1$	$\tau = 1{,}0$	$\alpha = 0{,}9$
Bernoulli	$\varepsilon = 0{,}0$	$\tau = 0{,}1$	$\alpha = 0{,}1$
Binomial	$\varepsilon = 0{,}1$	$\tau = 1{,}0$	$\alpha = 0{,}1$

Tabla 1.2: Resultados finales de la comparación entre políticas

Métrica	Distribución	ε-greedy	Softmax	UCB2
Recompensa promedio	Normal	8.913	8.342	9.514
	Bernoulli	0.960	0.861	0.951
	Binomial	7.151	6.776	7.763
% brazo óptimo	Normal	89.8	37.6	99.6
	Bernoulli	100.0	58.4	98.8
	Binomial	91.0	66.2	100.0
Regret acumulado	Normal	634.09	1206.37	44.58
	Bernoulli	0.18	97.31	8.72
	Binomial	634.63	1009.93	35.41

generales, UCB2 presentó el mejor desempeño en las distribuciones Normal y Binomial, mientras que en Bernoulli la mejor configuración correspondió a ε-greedy. Softmax, en cambio, quedó sistemáticamente por detrás en la comparación final. Estas diferencias no solo reflejan el mecanismo de decisión de cada política, sino también el efecto que tuvieron los hiperparámetros seleccionados sobre el equilibrio entre exploración y explotación.

Dado que el *regret* acumulado constituye la métrica más representativa del proceso de aprendizaje, en las Figuras 1.1, 1.2 y 1.3 se muestra su evolución temporal para las mejores configuraciones seleccionadas en cada distribución. Estas gráficas permiten observar no solo el valor final alcanzado, sino también la rapidez con la que cada estrategia se aproxima al mejor brazo disponible y la estabilidad que mantiene a lo largo del horizonte temporal considerado.

En la distribución Normal, mostrada en la Figura 1.1, UCB2 mantiene el crecimiento más bajo del *regret* durante prácticamente todo el proceso. Este comportamiento puede entenderse a partir del valor seleccionado para su hiperparámetro: en esta distribución la mejor configuración fue $\alpha = 0{,}9$, lo que implica épocas más largas antes de reevaluar la acción elegida. En un entorno donde las recompensas son continuas y el brazo óptimo logra distinguirse con relativa claridad, esta exploración por bloques permite que, una vez detectada una acción prometedora, la política permanezca más tiempo explotándola y reduzca así el costo acumulado de seguir explorando. En ε-greedy, la mejor configuración fue $\varepsilon = 0{,}1$, lo que indica que cierta exploración sostenida resultó útil para no quedar atrapado prematuramente, aunque esa misma exploración fija introduce un costo persistente al seguir seleccionando brazos subóptimos incluso cuando la información disponible ya es suficiente. En Softmax, la mejor configuración fue $\tau = 1{,}0$, es decir, una temperatura relativamente alta dentro de las evaluadas. Esto sugiere que en la distribución Normal una exploración más amplia fue preferible a una demasiado agresiva; sin embargo, incluso así la asignación probabilística siguió siendo demasiado dispersa en comparación con las otras políticas, dificultando una concentración rápida y sostenida en el brazo óptimo.

Figura 1.1: Regret acumulado promedio para las tres estrategias bajo distribución Normal.

En la distribución Bernoulli, mostrada en la Figura 1.2, el comportamiento cambia de forma marcada. Aquí la mejor configuración de ε-greedy fue precisamente $\varepsilon = 0{,}0$, lo que significa que, una vez realizada la fase inicial de prueba de los brazos, la política pasó a comportarse de forma puramente codiciosa. En esta distribución, donde las recompensas están acotadas a dos resultados posibles, una identificación temprana del mejor brazo tiene un efecto especialmente fuerte, porque la política puede concentrarse inmediatamente en su explotación sin seguir pagando el costo de explorar. Eso explica que ε-greedy alcance aquí el menor *regret* y la mayor selección del brazo óptimo. En UCB2, la mejor configuración fue $\alpha = 0{,}1$, lo que corresponde a épocas más cortas y reevaluaciones más frecuentes. Este resultado es coherente con el análisis previo de esta familia: en Bernoulli, una exploración demasiado prolongada puede mantener al algoritmo enganchado a una acción subóptima durante más tiempo del deseable, mientras que un α bajo permite corregir antes esas decisiones. Softmax, por su parte, obtuvo su mejor resultado con $\tau = 0{,}1$, lo que indica que en esta distribución resultó preferible una política mucho más concentrada. Aun así, su desempeño quedó por detrás del de las otras dos estrategias, lo que sugiere que incluso con una temperatura baja la exploración probabilística siguió retrasando la convergencia plena hacia el brazo óptimo.

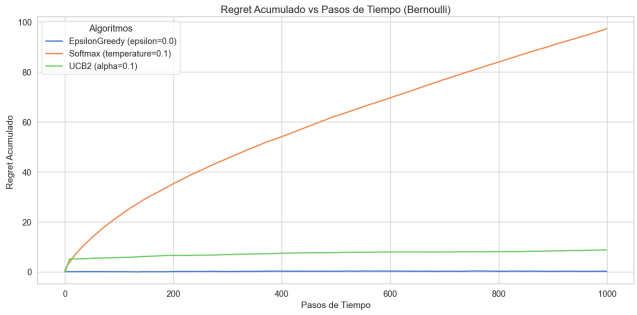

Figura 1.2: Regret acumulado promedio para las tres estrategias bajo distribución Bernoulli.

En la distribución Binomial, mostrada en la Figura 1.3, UCB2 vuelve a mostrar el mejor comportamiento global. En este caso, la mejor configuración fue nuevamente $\alpha = 0{,}1$, lo que indica que, bajo una distribución discreta con mayor variedad de valores posibles de recompensa, resulta ventajoso reeva-

luar con frecuencia y no prolongar demasiado las épocas de explotación temprana. Esta exploración estructurada permite distinguir con rapidez entre brazos con rendimientos cercanos y concentrarse después en el más prometedor sin incurrir en una exploración excesivamente aleatoria. Para ε-greedy, la mejor configuración volvió a ser $\varepsilon = 0{,}1$, lo que sugiere que en esta distribución la exploración persistente conserva utilidad para seguir refinando la estimación del brazo óptimo; sin embargo, ese mismo mecanismo continúa generando un costo acumulado mayor que el observado en UCB2. En Softmax, la mejor configuración fue nuevamente $\tau = 1{,}0$, como en la distribución Normal, lo que refuerza la idea de que las temperaturas más altas favorecieron en este estudio una exploración más útil que las opciones más bajas. No obstante, la política siguió mostrando una convergencia más lenta y una menor capacidad de concentración en la mejor acción disponible que las otras estrategias comparadas.

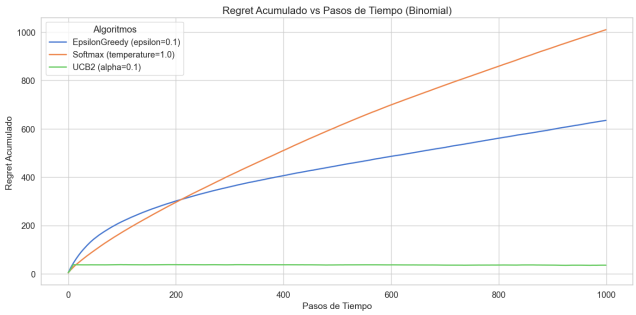

Figura 1.3: Regret acumulado promedio para las tres estrategias bajo distribución Binomial.

1.5. Conclusiones

En este capítulo se abordó el problema del bandido de k-brazos mediante la implementación y comparación de tres estrategias de selección de acciones: ε-greedy, Softmax y UCB. El análisis se realizó considerando distribuciones de recompensa de tipo Normal, Bernoulli y Binomial, y utilizando como métricas la recompensa promedio, el porcentaje de selección del brazo óptimo y el *regret* acumulado.

Los resultados obtenidos muestran que el desempeño de las políticas depende de la distribución de recompensa considerada y, en particular, de la forma en que cada estrategia regula el equilibrio entre exploración y explotación a través de sus hiperparámetros. En las distribuciones Normal y Binomial, la variante UCB2 presentó el mejor comportamiento global, lo que sugiere que su mecanismo de exploración estructurada resulta especialmente adecuado cuando conviene combinar exploración inicial y posterior concentración en el brazo más prometedor. En la distribución Bernoulli, en cambio, la estrategia ε-greedy con $\varepsilon = 0{,}0$ logró el mejor resultado, asociado a una identificación temprana del brazo óptimo y a su posterior explotación sostenida.

De forma general, UCB2 fue la estrategia más robusta en este estudio, ya que mostró el mejor desempeño en dos de las tres distribuciones analizadas y dominó con claridad en la métrica principal de evaluación. Por su parte, Softmax presentó sistemáticamente los resultados más débiles en la comparación final, lo que sugiere que, bajo las configuraciones seleccionadas, su exploración probabilística no favoreció una convergencia suficientemente rápida hacia la mejor acción disponible.

En conjunto, estos resultados permiten concluir que no existe una política que pueda considerarse universalmente superior, sino que su efectividad depende del esquema probabilístico de recompensas y del modo en que sus parámetros modulan la exploración y la explotación. Así, el problema del bandido de k-brazos constituye un marco adecuado para analizar comparativamente estrategias de decisión secuencial y comprender mejor las fortalezas y limitaciones de cada enfoque.

Como trabajo futuro, sería interesante extender los experimentos a otras distribuciones de recompensa y a distintas cantidades de pasos, con el fin de analizar la estabilidad de las políticas evaluadas en escenarios más diversos. Asimismo, podrían explorarse nuevas configuraciones de hiperparámetros mediante una búsqueda en grilla, permitiendo estudiar de forma más sistemática el efecto de parámetros como ε, τ, c y α sobre el desempeño de cada estrategia. Finalmente, también resultaría de interés incorporar otras estrategias de selección de acciones, como Thompson Sampling, para ampliar la comparación entre enfoques de exploración y explotación.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

El aprendizaje por refuerzo en entornos complejos extiende el marco estudiado en problemas más simples, como el bandido de k-brazos, hacia situaciones en las que un agente debe interactuar secuencialmente con un entorno, considerando que cada acción influye no solo en la recompensa inmediata, sino también en la evolución futura del sistema. En este contexto, el problema adquiere una mayor riqueza y dificultad, ya que el agente debe aprender políticas de decisión efectivas en presencia de dinámicas temporales, recompensas diferidas y espacios de estados que pueden crecer considerablemente. Esto resulta relevante porque permite modelar tareas más próximas a aplicaciones reales del aprendizaje por refuerzo, donde la toma de decisiones no puede reducirse a acciones aisladas, sino que requiere aprendizaje secuencial a partir de la interacción con el entorno y consideración de recompensas futuras [2].

La motivación de este trabajo radica en estudiar el comportamiento de distintos enfoques de aprendizaje por refuerzo ante problemas de complejidad creciente. En entornos con dinámicas secuenciales más ricas y espacios de estados más amplios, surgen nuevos desafíos para el diseño, entrenamiento y evaluación de los agentes [2].

El objetivo de este capítulo es presentar, desarrollar y analizar los experimentos realizados en entornos complejos, distinguiendo entre métodos tabulares y métodos aproximados. Para ello, se estudia el comportamiento de algoritmos tabulares en el entorno `Taxi-v3`, así como el desempeño de métodos aproximados en los entornos `CartPole-v1` y `Tetris`, todos ellos implementados bajo el esquema de interacción propuesto por *Gymnasium*. A partir de estos experimentos, se busca examinar las características de cada enfoque, comparar sus resultados y discutir sus ventajas, limitaciones y ámbitos de aplicación dentro del aprendizaje por refuerzo.

El resto del capítulo se organiza de la siguiente manera. En primer lugar, se presenta el desarrollo conceptual del problema, incluyendo su definición formal, la distinción entre métodos tabulares y aproximados, y una breve revisión de trabajos relacionados. A continuación, se describen los algoritmos utilizados en los experimentos, distinguiendo entre aquellos basados en representaciones tabulares y aquellos que emplean funciones aproximadoras para estimar los valores de estado o acción. Posteriormente, se expone la evaluación experimental, organizada en dos bloques principales: uno dedicado a los métodos tabulares y otro centrado en los métodos aproximados, con sus respectivos análisis de resultados. Finalmente, se presentan las conclusiones generales del capítulo, destacando los hallazgos más relevantes y posibles líneas de trabajo futuro.

2.2. Desarrollo

En esta sección se presenta el problema del aprendizaje por refuerzo en entornos complejos de manera más formal y se introducen algunos conceptos y antecedentes que permiten contextualizar su estudio. En particular, se distinguen los enfoques tabulares y aproximados, así como algunos trabajos relacionados relevantes para el desarrollo posterior del capítulo.

2.2.1. Definición formal del problema

El problema de aprendizaje por refuerzo puede modelarse formalmente mediante un Proceso de Decisión de Markov (MDP, por sus siglas en inglés), representado por la quíntupla

$$\mathcal{M} = \langle S, A, P, R, \gamma \rangle, \tag{2.1}$$

donde S es el conjunto de estados, A el conjunto de acciones, P la dinámica de transición del entorno, R la función de recompensa y $\gamma \in [0, 1]$ el factor de descuento [2, 7].

En este marco, el agente interactúa secuencialmente con el entorno. En cada instante temporal t, observa un estado $S_t \in S$, selecciona una acción $A_t \in A$, recibe una recompensa R_{t+1} y transita a un nuevo estado S_{t+1}. La dinámica de transición puede expresarse como

$$P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s,\ A_t = a), \tag{2.2}$$

mientras que la recompensa esperada asociada a una interacción puede definirse como

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s,\ A_t = a]. \tag{2.3}$$

El comportamiento del agente está determinado por una política π, que especifica la probabilidad de seleccionar una acción dada una observación del estado:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s). \tag{2.4}$$

A partir de esta interacción, el objetivo es maximizar el retorno acumulado esperado, definido como

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{2.5}$$

Bajo esta formulación, el aprendizaje por refuerzo busca estimar políticas o funciones de valor que permitan al agente actuar de manera cada vez más efectiva en el entorno [2, 7]. A partir de este marco general, a continuacións se distinguen dos enfoques principales utilizados en este trabajo: los métodos tabulares y los métodos aproximados.

2.2.2. Métodos tabulares

Los métodos tabulares constituyen uno de los enfoques más clásicos dentro del aprendizaje por refuerzo y resultan especialmente adecuados cuando el espacio de estados y acciones es discreto y de tamaño manejable. En este tipo de métodos, la información relevante del proceso de aprendizaje se representa de forma explícita mediante tablas, lo que permite estimar directamente funciones de valor o de valor–acción a partir de la experiencia acumulada.

Este enfoque ofrece una base clara para estudiar el comportamiento de distintos algoritmos de control, ya que facilita tanto la actualización iterativa de los valores estimados como el análisis de la convergencia y del desempeño del agente. En particular, dentro de esta familia se encuentran métodos basados en Monte Carlo y en diferencias temporales, que han sido fundamentales en el desarrollo del aprendizaje por refuerzo. En especial, los métodos de control tabular han tenido un papel central en la evolución del área [8].

En el presente trabajo, los métodos tabulares se aplican sobre un entorno discreto, lo que permite analizar su comportamiento en condiciones donde una representación explícita de los valores sigue siendo viable.

2.2.3. Métodos aproximados

Los métodos aproximados surgen como una extensión natural de los enfoques tabulares cuando el problema crece lo suficiente como para que una representación explícita de los valores deje de ser práctica. En estos casos, en lugar de almacenar la información en tablas, se recurre a funciones aproximadoras que permiten estimar funciones de valor o de valor–acción a partir de la experiencia [7].

Este enfoque resulta especialmente útil en problemas donde la cantidad de situaciones posibles que puede enfrentar el agente es demasiado grande como para representar todos los valores de manera exhaustiva. De este modo, los métodos aproximados amplían el alcance del aprendizaje por refuerzo hacia escenarios más exigentes, manteniendo como objetivo el aprendizaje de políticas efectivas a partir de la experiencia.

En el presente trabajo, los métodos aproximados se aplican en entornos de mayor complejidad, lo que permite analizar su comportamiento en situaciones donde una representación tabular deja de ser práctica.

2.2.4. Trabajos Relacionados

Los métodos tabulares constituyen uno de los antecedentes más importantes del Aprendizaje por Refuerzo. En problemas con espacios de estados y acciones discretos y de tamaño manejable, estos enfoques permiten representar explícitamente la función de valor o de valor–acción y aprender políticas de control efectivas a partir de la experiencia. En esta línea, Watkins (1989) y, posteriormente, Sutton y Barto (2018) consolidaron a algoritmos como SARSA y Q-Learning como referencias fundamentales dentro del área [8, 2].

Además de su relevancia teórica, los métodos tabulares también han sido utilizados en problemas aplicados. Por ejemplo, Komorowski et al. (2018) emplearon un enfoque basado en SARSA para aprender estrategias de tratamiento personalizadas en pacientes con sepsis en unidades de cuidados intensivos, mostrando cómo este tipo de métodos puede utilizarse en tareas de apoyo a la decisión clínica [10].

Sin embargo, a medida que los problemas aumentan en complejidad, la representación explícita mediante tablas deja de ser práctica. Esto ocurre, por ejemplo, cuando el número de estados crece demasiado o cuando las variables del entorno son continuas. Frente a esta limitación, comenzaron a desarrollarse métodos aproximados, en los que la función de valor ya no se almacena de forma exhaustiva, sino que se estima mediante un conjunto de parámetros, permitiendo además generalizar entre situaciones similares, tal como se describe en Sutton y Barto (2018) y en Silver (2015) [2, 7].

Un punto de inflexión en esta evolución fue el trabajo de Mnih et al. (2015), quienes propusieron Deep Q-Network (DQN), combinando Aprendizaje por Refuerzo con redes neuronales profundas para resolver tareas de control en entornos de alta dimensionalidad [9]. Más allá de este antecedente metodológico, los enfoques aproximados también han sido aplicados en problemas reales. En este sentido, Moreno-Malo et al. (2024) utilizaron Deep Q-learning para el control adaptativo de semáforos, mostrando mejoras en los tiempos de espera dentro de un entorno urbano simulado [11].

En este contexto, el presente trabajo no propone un nuevo algoritmo, sino una comparación experimental y sistemática de métodos tabulares y aproximados en entornos con distinto nivel de complejidad. A diferencia de los trabajos aplicados, aquí se consideran escenarios controlados que permiten aislar el efecto de cada enfoque y analizar de forma homogénea su comportamiento mediante un mismo protocolo experimental.

2.3. Algoritmos

En esta sección se presentan los algoritmos utilizados en el presente trabajo para resolver problemas de Aprendizaje por Refuerzo en entornos de distinta complejidad. Primero se abordan los métodos tabulares y luego los métodos aproximados, de acuerdo con el tipo de representación empleado en cada caso.

2.3.1. Algoritmos tabulares

Estos métodos resultan adecuados para problemas con espacios de estados y acciones discretos y de tamaño manejable, ya que representan de forma explícita la información de aprendizaje mediante tablas. En particular, se abordan métodos basados en Monte Carlo y en diferencias temporales, que constituyen algunos de los enfoques clásicos más importantes dentro del Aprendizaje por Refuerzo.

Monte Carlo on-policy

El método Monte Carlo on-policy estima la función de valor–acción a partir de episodios completos generados por la misma política que se desea evaluar y mejorar. Su idea central consiste en promediar los retornos observados luego de cada visita a un par estado–acción, sin utilizar *bootstrapping*. Por esta razón, se trata de un método adecuado para tareas episódicas, ya que la actualización se realiza una vez finalizado cada episodio [2].

El retorno observado desde el instante t se define como:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T \tag{2.6}$$

En este trabajo se utiliza la variante *every-visit*, en la cual el valor de un par estado–acción se estima promediando los retornos obtenidos en cada una de sus apariciones dentro de los episodios generados por la política seguida. De forma incremental, esta actualización puede escribirse como:

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)} \left(G_t - Q(s,a)\right) \tag{2.7}$$

donde $N(s,a)$ representa la cantidad de veces que el par estado–acción (s,a) ha sido considerado en la estimación.

Algorithm 4 Monte Carlo on-policy (every-visit)

```
Initialize Q(s,a) arbitrarily and N(s,a) ← 0 for all (s,a)
for each episode do
    Generate an episode following policy π
    Compute returns G_t for each step of the episode
    for each occurrence of a state-action pair (s,a) in the episode do
        N(s,a) ← N(s,a) + 1
        Q(s,a) ← Q(s,a) + 1/N(s,a) (G_t − Q(s,a))
    end for
    Improve policy π from Q (e.g., ε-greedy)
end for
```

Monte Carlo off-policy

Ya en el caso de Monte Carlo off-policy, este método permite aprender o evaluar una política objetivo a partir de episodios generados por una política de comportamiento diferente. En este trabajo se utiliza una política de comportamiento ϵ-greedy para favorecer la exploración, mientras que la política objetivo es greedy respecto a Q. Para corregir la diferencia entre ambas, se emplea *Weighted Importance Sampling*, lo que permite reponderar los retornos observados según la probabilidad relativa de las acciones bajo ambas políticas [2].

La razón de muestreo por importancia desde el instante t hasta el final del episodio se define como:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)} \tag{2.8}$$

donde π es la política objetivo y b la política de comportamiento. En la variante ponderada utilizada en este trabajo, la actualización incremental para la función de valor–acción puede expresarse como:

$$Q(s,a) \leftarrow Q(s,a) + \frac{W}{C(s,a)} \left(G - Q(s,a)\right) \tag{2.9}$$

donde W es el peso acumulado de importancia y $C(s,a)$ es el acumulador de pesos asociado al par estado–acción (s,a).

Algorithm 5 Monte Carlo off-policy with weighted importance sampling

```
Initialize Q(s,a) arbitrarily, C(s,a) ← 0 for all (s,a)
Let target policy π be greedy with respect to Q
for each episode do
    Generate an episode using behavior policy b
    Store (S_t, A_t, R_{t+1}, b(A_t | S_t))
    G ← 0, W ← 1
    for each step of the episode, in reverse order do
        G ← γG + R_{t+1}
        C(S_t, A_t) ← C(S_t, A_t) + W
        Q(S_t, A_t) ← Q(S_t, A_t) + W/C(S_t, A_t) (G − Q(S_t, A_t))
        Update π to be greedy with respect to Q
        if A_t ≠ π(S_t) then
            break
        end if
        W ← W/b(A_t | S_t)
    end for
end for
```

SARSA

Método de diferencias temporales *on-policy* que actualiza la función de valor–acción utilizando la recompensa observada y el valor estimado del siguiente par estado–acción. A diferencia de Monte Carlo, no necesita esperar al final del episodio para realizar la actualización, ya que emplea *bootstrapping*. En este trabajo se utiliza su versión tabular, adecuada para problemas con espacios de estados y acciones discretos y de tamaño manejable [2].

La actualización de SARSA se expresa como:

$$\begin{aligned} Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha \Big[& R_{t+1} \\ & + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t) \Big] \end{aligned} \tag{2.10}$$

donde α es la tasa de aprendizaje, γ es el factor de descuento y $Q(S_{t+1},A_{t+1})$ corresponde al valor estimado de la siguiente acción seleccionada bajo la misma política.

Algorithm 6 SARSA

Initialize $Q(s, a)$ arbitrarily for all (s, a)
for each episode **do**
  Initialize state S
  Choose action A from S using policy derived from Q (e.g., ϵ-greedy)
  while S is not terminal **do**
    Take action A
    Observe reward R and next state S'
    Choose action A' from S' using policy derived from Q
    $Q(S, A) \leftarrow Q(S, A) + \alpha\,[R + \gamma Q(S', A') - Q(S, A)]$
    $S \leftarrow S'$
    $A \leftarrow A'$
  end while
end for

Q-Learning

Otro método de diferencias temporales *off-policy* que actualiza la función de valor–acción utilizando la recompensa observada y el mayor valor estimado en el siguiente estado. A diferencia de SARSA, su actualización no depende de la acción efectivamente seleccionada por la política de comportamiento, sino de la mejor acción estimada en el estado siguiente. En este trabajo se utiliza su versión tabular, apropiada para problemas con espacios de estados y acciones discretos y de tamaño manejable [2].

La regla de actualización de Q-Learning se expresa como:

$$\begin{aligned} Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} \\ + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\Big] \end{aligned} \tag{2.11}$$

donde α es la tasa de aprendizaje, γ es el factor de descuento y el término $\max_{a'} Q(S_{t+1}, a')$ representa la mejor estimación disponible para el estado siguiente.

Algorithm 7 Q-Learning

Initialize $Q(s, a)$ arbitrarily for all (s, a)
for each episode **do**
  Initialize state S
  while S is not terminal **do**
    Choose action A from S using behavior policy derived from Q (e.g., ϵ-greedy)
    Take action A
    Observe reward R and next state S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\,[R + \gamma \max_{a'} Q(S', a') - Q(S, A)]$
    $S \leftarrow S'$
  end while
end for

2.3.2. Algoritmos aproximados

En esta sección se presentan los algoritmos aproximados utilizados en el presente trabajo para abordar problemas de Aprendizaje por Refuerzo en entornos de mayor complejidad. A diferencia de los métodos tabulares, estos enfoques estiman la función de valor o de valor–acción mediante funciones parametrizadas, lo que permite trabajar en problemas donde una representación explícita deja de ser práctica.

SARSA semi-gradiente

Aquí tenemos una extensión del algoritmo SARSA al caso en que la función de valor–acción no se representa mediante una tabla, sino a través de una función aproximada parametrizada. En este trabajo se utiliza su versión episódica *on-policy* de un paso, con aproximación lineal, lo que permite aplicar el método en entornos donde el espacio de estados es continuo o demasiado grande para una representación exhaustiva [2].

La función de valor–acción aproximada se expresa como:

$$Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \boldsymbol{\phi}(s) \tag{2.12}$$

donde $\boldsymbol{\phi}(s)$ es el vector de características del estado y $\mathbf{w}_a$ es el vector de pesos asociado a la acción a. En este caso, la actualización utilizada puede escribirse como:

$$\mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha\Big[r + \gamma Q(s', a'; \mathbf{w}) - Q(s, a; \mathbf{w})\Big]\boldsymbol{\phi}(s) \tag{2.13}$$

Esta expresión corresponde al caso de aproximación lineal, en el que la corrección de los pesos asociados a la acción seleccionada se realiza directamente a partir del error temporal y del vector de características del estado actual.

Algorithm 8 Semi-gradient SARSA

Initialize weight vectors $\mathbf{w}$ arbitrarily
for each episode **do**
  Initialize state S
  Compute feature vector $\boldsymbol{\phi}(S)$
  Choose action A using an ϵ-greedy policy derived from $Q(S, \cdot; \mathbf{w})$
  while S is not terminal **do**
    Take action A
    Observe reward R and next state S'
    Compute feature vector $\boldsymbol{\phi}(S')$
    Choose action A' using an ϵ-greedy policy derived from $Q(S', \cdot; \mathbf{w})$
    $\delta \leftarrow R + \gamma Q(S', A'; \mathbf{w}) - Q(S, A; \mathbf{w})$
    $\mathbf{w}_A \leftarrow \mathbf{w}_A + \alpha\,\delta\,\boldsymbol{\phi}(S)$
    $S \leftarrow S'$, $A \leftarrow A'$
  end while
end for

Deep Q-Network (DQN)

Este algoritmo es una extensión de Q-Learning en la que la función de valor–acción no se representa mediante una tabla, sino a través de una red neuronal profunda. Esto permite aplicar el algoritmo en problemas con espacios de estados de mayor dimensión, donde una representación tabular deja de ser práctica. En este trabajo se utiliza una implementación

basada en una red *online*, una red objetivo, memoria de repetición y política ϵ-greedy, siguiendo la línea propuesta por Mnih et al. (2015) [9].

La red aproxima la función de valor–acción como:

$$Q(s, a; \theta) \tag{2.14}$$

donde θ representa los parámetros de la red. A partir de un minibatch de transiciones almacenadas en la memoria de repetición, el valor objetivo utilizado en la actualización se define como:

$$y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\text{target}}(s'_i, a'; \theta^-) \tag{2.15}$$

donde d_i indica si la transición corresponde a un estado terminal y θ^- son los parámetros de la red objetivo. La función de pérdida utilizada en este trabajo es la pérdida de Huber:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\text{Huber}\big(Q(s_i, a_i; \theta) - y_i\big)\right] \tag{2.16}$$

Algorithm 9 Deep Q-Network (DQN)

```
Initialize online network Q(·, ·; θ)
Initialize target network Q_target(·, ·; θ⁻) with θ⁻ ← θ
Initialize replay buffer D
for each episode do
    Initialize state S
    while S is not terminal do
        Choose action A from S using an ϵ-greedy policy derived from Q(S, ·; θ)
        Take action A
        Observe reward R, next state S′ and terminal flag
        Store transition (S, A, R, S′, done) in D
        Sample a minibatch of transitions from D
        Compute targets using the target network
        Update θ by minimizing the Huber loss
        Every C steps, update θ⁻ ← θ
        Decay ϵ
        S ← S′
    end while
end for
```

2.4. Evaluación/Experimentos

Se presentan los experimentos realizados para evaluar el desempeño de los algoritmos considerados en el trabajo. En primer lugar, se analizan los métodos tabulares en un entorno discreto, comparando su comportamiento bajo un mismo protocolo experimental. Luego, se estudian los métodos aproximados en entornos de mayor complejidad, con el objetivo de observar sus capacidades y limitaciones en problemas donde una representación tabular deja de ser práctica.

2.4.1. Métodos tabulares

Para el estudio de los métodos tabulares se utilizó el entorno *Taxi-v3*, que plantea un problema discreto de control con recompensas bien definidas y episodios relativamente cortos. Sobre este entorno se evaluaron Monte Carlo on-policy, Monte Carlo off-policy, SARSA y Q-Learning bajo un mismo protocolo experimental, con el fin de comparar su comportamiento en condiciones homogéneas.

Entorno y protocolo experimental

Para evaluar los métodos tabulares se utilizó el entorno `Taxi-v3` de Gymnasium, un problema discreto de control con 500 estados y 6 acciones. En este entorno, el agente debe recoger a un pasajero en una ubicación determinada y dejarlo en su destino correcto. La recompensa penaliza cada paso con -1, asigna $+20$ cuando la tarea se completa correctamente y aplica -10 ante acciones ilegales. Por sus características, se trata de un entorno apropiado para estudiar métodos tabulares mediante una representación explícita de la función de valor–acción.

Los cuatro algoritmos considerados —Monte Carlo *on-policy*, Monte Carlo *off-policy* con *Weighted Importance Sampling*, SARSA y Q-Learning— se evaluaron bajo un mismo protocolo experimental. En todos los casos se utilizaron las semillas $\{123, 231, 777, 2024, 31415\}$, 10000 episodios de entrenamiento por semilla y un factor de descuento $\gamma = 0{,}99$. La exploración se implementó con una política ε-greedy, con ε inicial de 1,0, mínimo de 0,01 y decaimiento multiplicativo de 0,9995. En los métodos de diferencias temporales se fijó además una tasa de aprendizaje $\alpha = 0{,}1$.

Para distinguir el comportamiento exploratorio durante el entrenamiento de la calidad real de la política aprendida, se realizaron evaluaciones *greedy* cada 250 episodios, utilizando 50 episodios por evaluación. Al finalizar, cada agente fue evaluado nuevamente durante 200 episodios *greedy* por semilla. Además, para resumir el comportamiento reciente del entrenamiento se consideraron los últimos 1000 episodios y, en la visualización de las curvas, se aplicó una ventana de suavizado de 200 episodios.

El criterio de éxito se definió a partir de la terminación real del entorno. En esta implementación, un episodio se considera exitoso cuando finaliza con `terminated`, mientras que los casos cerrados por truncamiento se contabilizan como no exitosos. Esto evita identificar el éxito únicamente mediante `reward >0`, ya que en `Taxi-v3` esa condición no siempre representa correctamente la finalización satisfactoria de la tarea. A partir de este protocolo, se analizaron como métricas principales la recompensa media, la longitud media del episodio y la tasa de éxito en evaluación *greedy*.

Resultados de Monte Carlo *on-policy*

En el caso de Monte Carlo *on-policy* (*every-visit*), la política *greedy* final obtuvo un *reward* medio de $-136{,}422 \pm 16{,}990$, con una longitud media de 142.8 pasos y una tasa de éxito de 30.4 %. Aunque se observa una mejora sostenida durante el entrenamiento, el método no alcanzó *reward* positivo ni logró superar el 50 % de éxito bajo el presupuesto experimental utilizado, por lo que su desempeño sigue siendo claramente insuficiente en `Taxi-v3`.

En las evaluaciones *greedy* periódicas, el *reward* medio pasó

de $-786{,}7$ en el episodio 250 a $-128{,}9$ en el episodio 10000, mientras que la tasa de éxito aumentó de 0.4 % a 34.0 %. Aun así, la evaluación final confirma que la política aprendida permanece lejos de una solución competitiva. Además, la diferencia entre las métricas de los últimos episodios de entrenamiento y las obtenidas en evaluación *greedy* sugiere una dependencia residual de la política ε-greedy utilizada durante el aprendizaje.

Este resultado es coherente con la naturaleza del método, ya que las actualizaciones de la tabla Q se realizan únicamente al finalizar cada episodio. En un entorno como `Taxi-v3`, esto enlentece la propagación de información útil y limita la velocidad de aprendizaje frente a enfoques basados en diferencias temporales.

Resultados de Monte Carlo off-policy

En Monte Carlo *off-policy* con *Weighted Importance Sampling*, la política *greedy* final obtuvo un *reward* medio de $-4{,}953 \pm 5{,}543$, con una longitud media de 23.154 pasos y una tasa de éxito de 95.2 %. Aunque el porcentaje de éxito es alto, el *reward* permanece levemente negativo, lo que indica que la política aprendida todavía resuelve muchos episodios con trayectorias más largas de lo deseable.

Durante el entrenamiento se observa una mejora clara en las evaluaciones *greedy*: el *reward* medio pasa de $-200{,}0$ en el episodio 250 a $-12{,}9$ en el episodio 10000, mientras que la tasa de éxito aumenta de 0.0 % a 94.0 %. El método alcanza 50 %, 75 % y 90 % de éxito en los episodios 3750, 5250 y 7750, respectivamente, aunque no llega al 99 % dentro del horizonte considerado.

La evaluación final resulta además coherente con el comportamiento observado en entrenamiento. En los últimos 1000 episodios se obtuvo un *reward* medio de $-7{,}678 \pm 4{,}551$, una longitud media de 25.580 pasos y una tasa de éxito de 93.9 %, valores próximos a los de la evaluación *greedy* final. En conjunto, estos resultados muestran un avance claro respecto a Monte Carlo *on-policy*, aunque todavía por debajo del nivel alcanzado por los métodos basados en diferencias temporales.

Resultados de SARSA

En SARSA, la política *greedy* final obtuvo un *reward* medio de $7{,}722 \pm 0{,}341$, con una longitud media de 13.278 pasos y una tasa de éxito de 100.0 %. Estos resultados muestran que el método logra aprender una política eficaz en `Taxi-v3`, con episodios cortos, alta estabilidad y resolución consistente de la tarea.

La evolución durante el entrenamiento también refleja una mejora sostenida. En las evaluaciones *greedy*, el *reward* medio pasó de $-200{,}0$ en el episodio 250 a 7,7 en el episodio 10000, mientras que la tasa de éxito aumentó de 0.0 % a 100.0 %. Además, el método alcanzó 50 %, 75 %, 90 % y 99 % de éxito en los episodios 2000, 2500, 3250 y 4500, respectivamente, lo que evidencia una convergencia mucho más rápida que la observada en ambos métodos Monte Carlo.

La evaluación final resulta coherente con el comportamiento reciente del entrenamiento. En los últimos 1000 episodios se obtuvo un *reward* medio de $7{,}499 \pm 0{,}362$, una longitud media de 13.501 pasos y una tasa de éxito de 100.0 %, valores muy próximos a los medidos en evaluación *greedy*. En conjunto, esto indica que SARSA aprende una política estable y bien adaptada al entorno bajo el protocolo experimental considerado.

Resultados de Q-Learning

En Q-Learning, la política *greedy* final obtuvo un *reward* medio de $7{,}788 \pm 0{,}246$, con una longitud media de 13.212 pasos y una tasa de éxito de 100.0 %. Estos resultados indican que el método alcanza un desempeño excelente en `Taxi-v3`, con episodios breves, alta estabilidad y resolución consistente de la tarea.

La evolución durante el entrenamiento muestra una mejora rápida y sostenida. En las evaluaciones *greedy*, el *reward* medio pasó de $-200{,}0$ en el episodio 250 a 7,8 en el episodio 10000, mientras que la tasa de éxito aumentó de 0.0 % a 100.0 %. Además, el método alcanzó 50 %, 75 %, 90 % y 99 % de éxito en los episodios 1750, 2250, 2750 y 4000, respectivamente, lo que refleja una convergencia ligeramente más rápida que la observada en SARSA.

La evaluación final resulta coherente con el comportamiento reciente del entrenamiento. En los últimos 1000 episodios se obtuvo un *reward* medio de $7{,}591 \pm 0{,}284$, una longitud media de 13.409 pasos y una tasa de éxito de 100.0 %, valores muy próximos a los de la evaluación *greedy*. En conjunto, esto muestra que Q-Learning fue el método con mejor desempeño global dentro del conjunto tabular evaluado.

Comparativa global de métodos tabulares

La Tabla 2.1 resume los resultados finales obtenidos por los cuatro métodos tabulares en `Taxi-v3`, mientras que la Figura 2.1 muestra la evolución de las principales métricas a lo largo del entrenamiento. En conjunto, ambos elementos permiten observar diferencias claras tanto en la velocidad de aprendizaje como en la calidad final de las políticas obtenidas.

Tabla 2.1: Resultados finales de la comparación entre métodos tabulares en Taxi-v3

Métrica	MC On-Pol.	MC Off-Pol.	SARSA	Q-Learn.
Reward final	-136.422	-4.953	7.722	**7.788**
Longitud final	142.806	23.154	13.278	**13.212**
Éxito final (%)	30.4	95.2	**100.0**	**100.0**

Los resultados muestran una separación marcada entre los métodos Monte Carlo y los métodos basados en diferencias temporales. Monte Carlo *on-policy* fue claramente el enfoque más débil, con *reward* final negativo, episodios largos y una tasa de éxito muy por debajo del resto. Monte Carlo *off-policy* logró una mejora importante respecto a esa variante, alcanzando 95.2 % de éxito, aunque todavía con un *reward* ligeramente negativo y trayectorias más extensas que las observadas en SARSA y Q-Learning.

Por su parte, los métodos de diferencias temporales obtuvieron los mejores resultados globales. Tanto SARSA como

Q-Learning alcanzaron 100.0 % de éxito en evaluación *greedy*, con *reward* positivo y longitudes medias cercanas a 13 pasos. Entre ambos, Q-Learning presentó el mejor *reward* final y la menor longitud media del episodio, además de una convergencia ligeramente más rápida, como puede apreciarse en la Figura 2.1. En términos generales, la comparación confirma que, bajo el protocolo experimental considerado, los métodos TD resultan más eficaces y eficientes que los enfoques Monte Carlo para este entorno.

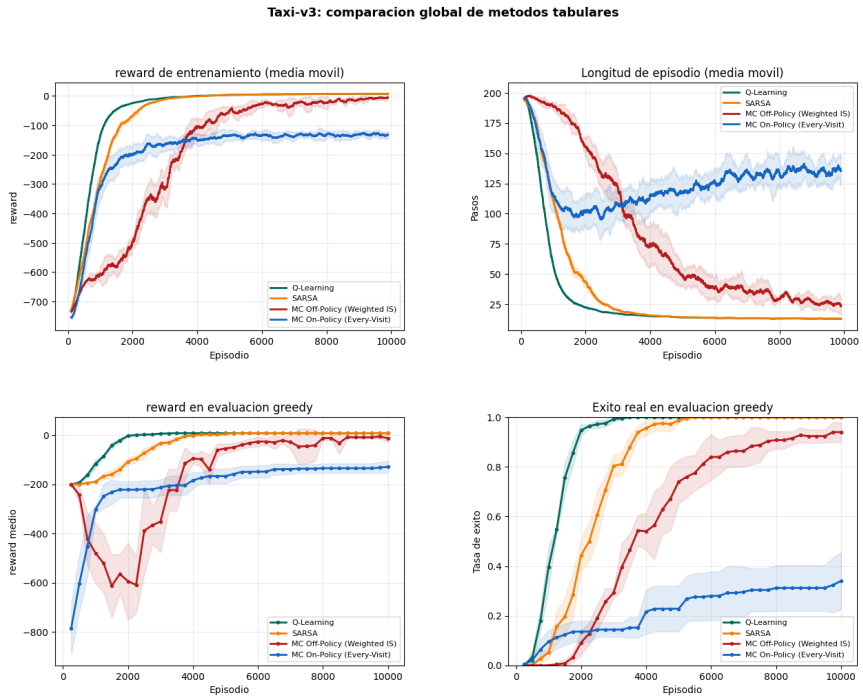

Figura 2.1: Comparación global de los métodos tabulares evaluados en `Taxi-v3`.

Análisis comparativo de resultados

Los resultados obtenidos muestran con bastante claridad que, en `Taxi-v3`, los métodos basados en diferencias temporales se adaptan mejor que los enfoques Monte Carlo al presupuesto experimental considerado. Tanto SARSA como Q-Learning lograron aprender políticas efectivas, con *reward* positivo, episodios cortos y una tasa de éxito del 100 % en evaluación *greedy*. En cambio, los métodos Monte Carlo presentaron una convergencia más lenta y un desempeño final inferior, especialmente en la variante *on-policy*.

Desde el punto de vista del aprendizaje, esta diferencia resulta consistente con la forma en que cada familia de métodos actualiza la función de valor–acción. En Monte Carlo, las actualizaciones se realizan únicamente al finalizar cada episodio, por lo que la información útil tarda más en propagarse hacia los pares estado–acción relevantes. Esto penaliza especialmente a Monte Carlo *on-policy*, que bajo este protocolo no logró alcanzar una política competitiva. Monte Carlo *off-policy*, al incorporar *Weighted Importance Sampling*, consiguió una mejora importante, aunque todavía quedó por debajo de los métodos TD en términos de *reward* y longitud de trayectoria.

Entre los métodos de diferencias temporales, ambos mostraron un comportamiento muy sólido, aunque Q-Learning presentó una ligera ventaja sobre SARSA tanto en velocidad de convergencia como en desempeño final. Esto puede interpretarse a partir de su carácter *off-policy*, ya que actualiza hacia la mejor acción estimada disponible y tiende a aproximarse más rápidamente a una política codiciosa. SARSA, por su parte, al ser *on-policy*, incorpora en la actualización el efecto de la exploración y por ello suele mantener una evolución algo más conservadora, aunque en este entorno también alcanzó resultados prácticamente óptimos.

En conjunto, el estudio confirma que `Taxi-v3` constituye un entorno adecuado para comparar métodos tabulares y que, dentro de este grupo, los enfoques TD ofrecen la mejor combinación entre rapidez de aprendizaje, estabilidad y calidad final de la política. Bajo las condiciones ensayadas, Q-Learning fue el método más efectivo, seguido muy de cerca por SARSA.

2.4.2. Métodos aproximados

El análisis de los métodos aproximados se realizó en entornos de mayor complejidad, donde una representación tabular deja de ser práctica. En este contexto se evaluaron SARSA semi-gradiente y Deep Q-Network (DQN), con el objetivo de observar su comportamiento en problemas con espacios de estados más exigentes y comparar sus capacidades bajo un mismo esquema general de experimentación.

Infraestructura y decisiones de implementación

Los experimentos de control aproximado se implementaron en `Python` utilizando `Gymnasium` para los entornos, `PyTorch` para los aproximadores y `NumPy`/`Pandas` para el procesamiento y agregación de resultados. El cuaderno se organizó de forma modular, separando la configuración experimental, la ejecución de los agentes y el análisis posterior.

Con el fin de sostener una comparación homogénea, se definió un protocolo común por entorno. En el modo de ejecución completo se utilizaron las semillas de entrenamiento $\{0, 1, 2, 3, 4\}$, con 500 episodios para `CartPole-v1` y 250 pa-

ra Tetris. La evaluación final se realizó de forma separada y *greedy*, usando 25 semillas en `CartPole-v1` y 20 en Tetris. Además, para resumir el comportamiento reciente del entrenamiento se consideraron ventanas finales de 50 episodios en `CartPole-v1` y 40 en Tetris, junto con una ventana de suavizado de 30 episodios para la visualización de las curvas.

También se procuró compartir la representación de entrada dentro de cada entorno, de modo que la comparación quedara centrada en el algoritmo y en el tipo de aproximador. En `CartPole-v1`, tanto SARSA semi-gradiente como DQN operan sobre una base de Fourier de orden 2, lo que produce 81 características. En Tetris, ambos métodos reciben el mismo vector de 7 rasgos artesanales del tablero. Asimismo, en ambos algoritmos se utilizó una política ε-greedy con ε inicial igual a 1.0; en `CartPole-v1` se fijó $\varepsilon_{\text{mín}} = 0{,}01$ y en Tetris $\varepsilon_{\text{mín}} = 0{,}05$, manteniendo un calendario de exploración alineado entre métodos.

En cuanto a las decisiones específicas de implementación, SARSA se resolvió con aproximación lineal semi-gradiente y $\gamma = 0{,}99$, utilizando $\alpha = 3\times10^{-3}$ en `CartPole-v1` y $\alpha = 10^{-2}$ en Tetris. DQN se implementó con una red neuronal multicapa, memoria de repetición y red objetivo, también con $\gamma = 0{,}99$. En `CartPole-v1` se emplearon dos capas ocultas de 64 unidades, tasa de aprendizaje de 5×10^{-4}, *batch size* de 64, *replay buffer* de 15000 transiciones y actualización de la red objetivo cada 120 pasos. En Tetris se utilizaron dos capas ocultas de 96 unidades, tasa de aprendizaje de 7×10^{-4}, el mismo tamaño de lote y de buffer, y actualización objetivo cada 180 pasos.

Finalmente, el protocolo separa de forma explícita entrenamiento y evaluación, de modo que el rendimiento final de cada agente se mide con $\varepsilon = 0$. Esto permite distinguir el comportamiento exploratorio durante el aprendizaje de la calidad real de la política aprendida y comparar los métodos bajo un criterio consistente.

SARSA semi-gradiente

En `CartPole-v1`, SARSA semi-gradiente alcanzó un retorno medio final de $500{,}00 \pm 0{,}00$ en evaluación *greedy* y $500{,}00 \pm 0{,}00$ en la ventana final de entrenamiento, lo que equivale al máximo teórico del entorno. La dinámica observada en las curvas muestra además una convergencia temprana y sostenida: la media suavizada supera la referencia histórica de entorno resuelto (195) en el episodio 110 y entra en la franja casi óptima de 475 puntos en el episodio 179, mientras que el mejor tramo suavizado se localiza en el episodio 228. En este caso, la coincidencia exacta entre entrenamiento terminal y evaluación final, junto con la desviación nula entre semillas, indica que el método aprende una política altamente estable y prácticamente saturada bajo el protocolo considerado.

En Tetris, SARSA también mostró un comportamiento sólido y muy estable. La política *greedy* final obtuvo $9{,}950 \pm 0{,}000$, mientras que en la ventana final de entrenamiento se registró $9{,}825 \pm 0{,}077$. Dado que en este entorno no se adopta un umbral externo de resolución, la interpretación se apoya en el nivel final alcanzado, la estabilidad entre semillas y la coherencia entre entrenamiento y evaluación. En este sentido, la curva muestra una mejora gradual que termina estabilizándose en una meseta alta y estrecha; la media final de entrenamiento mejora en 0,130 puntos respecto a la ventana inicial y el mejor tramo suavizado aparece en el episodio 238. La diferencia pequeña entre entrenamiento y evaluación, junto con la ausencia de dispersión en la política final, sugiere que el aproximador lineal logra extraer una política reproducible y consistente también en este entorno.

Deep Q-Network (DQN)

En `CartPole-v1`, DQN obtuvo un retorno medio final de $468{,}73 \pm 60{,}94$ en evaluación *greedy* y $444{,}22 \pm 79{,}62$ en la ventana final de entrenamiento. El método supera con claridad la referencia histórica de entorno resuelto (195), por lo que puede considerarse exitoso en promedio; sin embargo, queda por debajo de SARSA tanto en rendimiento final como en estabilidad entre semillas. La dinámica observada en las curvas también muestra una convergencia más tardía: la media suavizada cruza la barrera de 195 en el episodio 144, pero no alcanza la franja casi óptima de 475 dentro del horizonte de entrenamiento. Además, el mejor tramo suavizado aparece recién en el episodio 499, lo que sugiere una evolución menos uniforme y más dependiente de la variabilidad entre corridas.

En términos de coherencia entre entrenamiento y evaluación, la política *greedy* final mejora en 24,51 puntos respecto al promedio terminal de entrenamiento. Esto indica que una parte del rendimiento observado al final del aprendizaje seguía condicionada por la exploración residual de la política ε-greedy, y que la red había aprendido una política explotativa mejor que la reflejada por las recompensas de entrenamiento. Aun así, la dispersión final sigue siendo apreciable, por lo que el comportamiento de DQN en `CartPole-v1` resulta correcto pero menos estable que el de SARSA bajo el protocolo considerado.

En Tetris, DQN mostró un comportamiento mucho más sólido y cercano al de SARSA. La política *greedy* final alcanzó $9{,}950 \pm 0{,}000$, mientras que la ventana final de entrenamiento registró $9{,}805 \pm 0{,}068$. A la precisión reportada, este resultado empata con SARSA en calidad final de la política aprendida, por lo que no se observa una ventaja final atribuible a la no linealidad del aproximador en este entorno. La curva de aprendizaje presenta una mejora gradual, con una ganancia de 0,145 puntos respecto a la ventana inicial y con el mejor tramo suavizado en el episodio 247, lo que describe una convergencia estable hacia una meseta final alta.

La diferencia pequeña entre entrenamiento y evaluación, junto con la ausencia de dispersión en la política *greedy* final, sugiere que DQN logra en Tetris una solución reproducible y consistente. En conjunto, los resultados muestran un contraste interesante entre entornos: en `CartPole-v1`, DQN resuelve el problema pero queda por debajo de SARSA en rapidez y estabilidad, mientras que en Tetris ambos métodos terminan prácticamente empatados en desempeño final.

Comparativa global de métodos aproximados

La Tabla 2.2 resume los resultados finales obtenidos por SARSA semi-gradiente y DQN en `CartPole-v1` y Tetris, mientras que la Figura 2.2 permite visualizar conjuntamente la evolución del entrenamiento y el rendimiento *greedy* final en ambos entornos. Los mejores valores se indican en negrita; en el caso

de la desviación estándar y del episodio en que se alcanza el mejor tramo suavizado, un valor menor se interpreta como mejor desempeño. En conjunto, ambos elementos permiten comparar no solo la calidad final de las políticas aprendidas, sino también la rapidez de aprendizaje y la estabilidad entre semillas.

Tabla 2.2: Resultados finales de la comparación entre métodos aproximados

Métrica	Entorno	SARSA semi-grad.	DQN
Eval greedy final	CartPole-v1	**500.00**	468.73
Eval greedy std	CartPole-v1	**0.00**	60.94
Train media final	CartPole-v1	**500.00**	444.22
Mejor ep. (MA-30)	CartPole-v1	**228**	499
Eval greedy final	Tetris	9.950	9.950
Eval greedy std	Tetris	0.000	0.000
Train media final	Tetris	**9.825**	9.805
Mejor ep. (MA-30)	Tetris	**238**	247

Los resultados muestran un comportamiento distinto según el entorno considerado. En `CartPole-v1`, SARSA semi-gradiente domina con claridad a DQN: alcanza el retorno máximo teórico de 500,00 en evaluación *greedy*, con desviación nula entre semillas, mientras que DQN obtiene 468,73±60,94. Además, SARSA converge antes y entra en una región de rendimiento casi óptimo dentro del horizonte de entrenamiento, mientras que DQN resuelve el entorno en promedio, pero con una dinámica más tardía y una variabilidad final considerablemente mayor.

En Tetris, en cambio, la comparación final resulta mucho más equilibrada. Ambos métodos alcanzan exactamente 9,950 ± 0,000 en evaluación *greedy* a la precisión reportada, por lo que no se observa una ventaja final de uno sobre otro en términos de calidad de la política aprendida. Las diferencias restantes se reducen a matices en la dinámica del entrenamiento: SARSA presenta una media final ligeramente superior (9,825 frente a 9,805) y alcanza su mejor tramo suavizado un poco antes, aunque estas diferencias no alteran la conclusión principal de empate práctico en desempeño final.

La Figura 2.2 refuerza esta lectura. En `CartPole-v1`, la ventaja de SARSA se aprecia tanto en la pendiente inicial de aprendizaje como en la estabilidad de la meseta final. En Tetris, por el contrario, ambas curvas terminan concentrándose en una región de rendimiento muy próxima, y las barras de evaluación final muestran que la diferencia entre métodos desaparece al considerar la política *greedy* aprendida. En conjunto, la comparación sugiere que, bajo este protocolo experimental, SARSA semi-gradiente ofrece una solución más robusta en `CartPole-v1`, mientras que en Tetris ninguno de los dos métodos presenta una superioridad final observable.

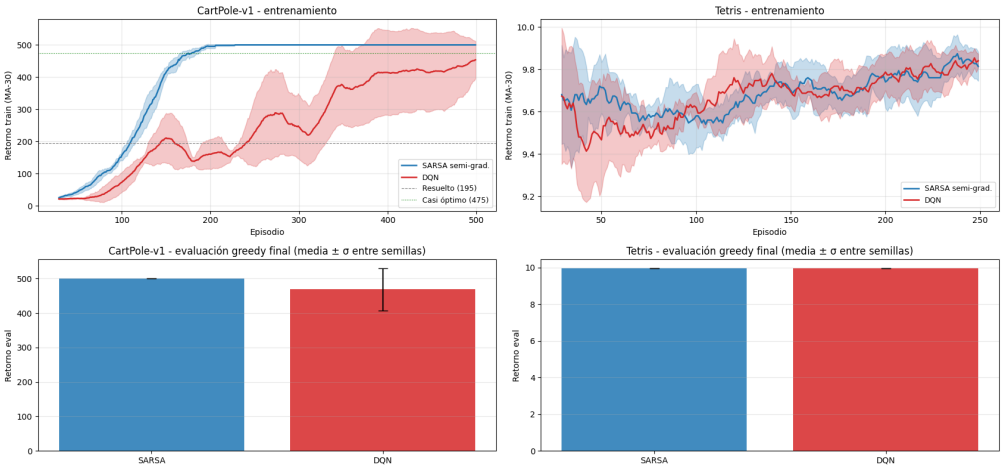

Figura 2.2: Comparación global entre SARSA semi-gradiente y DQN en `CartPole-v1` y Tetris. La fila superior muestra la evolución del retorno de entrenamiento y la fila inferior resume la evaluación *greedy* final.

Análisis comparativo de resultados

Los resultados obtenidos muestran que el comportamiento relativo de los métodos aproximados depende del entorno considerado. En `CartPole-v1`, SARSA semi-gradiente superó con claridad a DQN tanto en rendimiento final como en estabilidad. No solo alcanzó el retorno máximo teórico del entorno, sino que además lo hizo con dispersión nula entre semillas y con una convergencia más temprana. En cambio, aunque DQN logró resolver el entorno en promedio, presentó una variabilidad final considerablemente mayor y una evolución menos uniforme durante el entrenamiento.

Esta diferencia puede interpretarse a partir de la relación entre el entorno y el tipo de aproximador utilizado. En `CartPole-v1`, la representación mediante base de Fourier parece capturar de forma muy adecuada la estructura del problema, permitiendo que el aproximador lineal de SARSA aprenda una política prácticamente óptima de manera rápida y estable. En este contexto, la mayor complejidad de DQN no se traduce en una ventaja observable; por el contrario, introduce una dinámica de entrenamiento más sensible a la variabilidad propia de la optimización no lineal.

En Tetris, la situación fue distinta. Allí ambos métodos alcanzaron el mismo rendimiento final en evaluación *greedy* a la precisión reportada, lo que sugiere que, bajo el protocolo experimental utilizado, tanto el aproximador lineal como la red neuronal fueron capaces de explotar con éxito la representación de entrada disponible. Las diferencias observadas se limitaron a matices menores en la evolución del entrenamiento, con una ligera ventaja de SARSA en la media final y en la rapidez para alcanzar su mejor tramo suavizado, aunque sin impacto apreciable sobre la política final.

En conjunto, estos resultados indican que una arquitectura más compleja no garantiza por sí misma un mejor desempeño. Cuando la representación del estado es suficientemente informativa y el problema puede capturarse bien con un aproximador lineal, SARSA semi-gradiente puede ofrecer una combinación muy favorable de estabilidad, rapidez de aprendizaje y calidad final. Bajo las condiciones ensayadas, SARSA fue el método más sólido en `CartPole-v1`, mientras que en Tetris ambos enfoques mostraron un desempeño final prácticamente equivalente.

2.5. Conclusiones

En este capítulo se estudió el aprendizaje por refuerzo en entornos complejos a partir de dos grandes familias de métodos: los enfoques tabulares y los aproximados. En conjunto, los experimentos permitieron analizar cómo cambian las capacidades y limitaciones de los algoritmos cuando se pasa de un entorno discreto y manejable, como `Taxi-v3`, a problemas donde una representación tabular deja de ser práctica, como `CartPole-v1` y Tetris.

En el bloque de métodos tabulares, los resultados mostraron una diferencia clara entre los enfoques Monte Carlo y los métodos basados en diferencias temporales. Tanto SARSA como Q-Learning lograron aprender políticas eficaces en `Taxi-v3`, con *reward* positivo, episodios cortos y tasa de éxito del 100 % en evaluación *greedy*. En cambio, Monte Carlo *on-policy* presentó una convergencia demasiado lenta y un desempeño final insuficiente, mientras que Monte Carlo *off-policy* logró una mejora importante, aunque todavía por debajo de los métodos TD. Dentro de este grupo, Q-Learning fue el método de mejor desempeño global, seguido muy de cerca por SARSA.

En los métodos aproximados, el comportamiento dependió más fuertemente del entorno. En `CartPole-v1`, SARSA semi-gradiente superó con claridad a DQN, alcanzando el retorno máximo teórico del problema con mayor estabilidad y convergencia más temprana. En este caso, la combinación entre representación mediante base de Fourier y aproximación lineal resultó especialmente efectiva. En Tetris, en cambio, ambos métodos alcanzaron un desempeño final prácticamente equivalente en evaluación *greedy*, lo que sugiere que, bajo la representación utilizada, tanto el aproximador lineal como la red neuronal fueron capaces de extraer políticas finales de calidad comparable.

De forma general, los resultados del capítulo muestran que no siempre un método más complejo conduce a un mejor desempeño. En problemas donde el espacio de estados y acciones sigue siendo manejable, los métodos tabulares continúan siendo una alternativa muy eficaz y transparente. Del mismo modo, en entornos más complejos, un aproximador lineal bien diseñado puede igualar o incluso superar a una arquitectura más sofisticada si la representación del estado es suficientemente informativa. En este sentido, el estudio confirma que la elección del algoritmo debe analizarse en función del entorno, de la representación disponible y del equilibrio buscado entre estabilidad, rapidez de aprendizaje y calidad final de la política.

Como trabajo futuro, sería interesante ampliar la comparación a otros entornos de control con distinta dificultad, incluyendo problemas continuos y dominios con recompensas más dispersas o diferidas. También resultaría valioso estudiar con mayor profundidad el efecto de los hiperparámetros sobre la convergencia, por ejemplo mediante búsquedas sistemáticas que permitan contrastar configuraciones de exploración, tasas de aprendizaje, arquitecturas y tamaños de memoria de repetición. Finalmente, una extensión natural de este trabajo sería incorporar variantes más avanzadas, como Expected SARSA, Double DQN o métodos basados en actor-crítico, con el fin de ampliar la comparación entre enfoques y analizar hasta qué punto las conclusiones observadas aquí se mantienen en escenarios más exigentes.

Bibliografía

[1] Lattimore, T., y Szepesvári, C. (2020). *Bandit Algorithms*. Cambridge University Press.

[2] Sutton, R. S., y Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). The MIT Press.

[3] Auer, P., Cesa-Bianchi, N., y Fischer, P. (2002). *Finite-time Analysis of the Multiarmed Bandit Problem*. Machine Learning, 47(2–3), 235–256.

[4] Li, L., Chu, W., Langford, J., y Schapire, R. E. (2010). *A Contextual-Bandit Approach to Personalized News Article Recommendation*. Proceedings of the 19th International Conference on World Wide Web, 661–670.

[5] Bouneffouf, D., y Rish, I. (2019). *A Survey on Practical Applications of Multi-Armed and Contextual Bandits*. arXiv preprint arXiv:1904.10040.

[6] Villar, S. S., Bowden, J., y Wason, J. (2015). *Multi-armed Bandit Models for the Optimal Design of Clinical Trials: Benefits and Challenges*. Statistical Science, 30(2), 199–215.

[7] Silver, D. (2015). *Lectures on Reinforcement Learning*. University College London (UCL).

[8] Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards*. PhD thesis, University of Cambridge.

[9] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., y Hassabis, D. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529–533.

[10] Komorowski, M., Celi, L. A., Badawi, O., Gordon, A. C., y Faisal, A. A. (2018). *The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care*. Nature Medicine, 24(11), 1716–1720.

[11] Moreno-Malo, J., Posadas-Yagüe, J.-L., Cano, J. C., Calafate, C. T., Conejero, J. A., y Poza-Lujan, J.-L. (2024). *Improving traffic light systems using Deep Q-networks*. Expert Systems with Applications, 252(B), 124178.